Brief Announcement: Computing Least Fixed Points with Overwrite Semantics in Parallel and Distributed Systems

Vijay K. Garg
University of Texas at Austin
Department of Electrical and Computer Engineering
Austin, Texas, USA
garg@ece.utexas.edu

Rohan Garg
Purdue University
Department of Computer Science
West Lafayette, Indiana, USA
rohang@purdue.edu

Abstract

We present methods to compute least fixed points of multiple monotone inflationary functions in parallel and distributed settings. While the classic Knaster–Tarski theorem addresses a single function with sequential iteration, modern computing systems require parallel execution with overwrite semantics, non-atomic updates, and stale reads. We prove three convergence theorems under progressively relaxed synchronization: (1) Interleaving semantics with fair scheduling, (2) Parallel execution with update-only-on-change semantics (processes write only on those coordinates whose values change), and (3) Distributed execution with bounded staleness (updates propagate within T rounds) and i-locality (each process modifies only its own component).

Our approach differs from prior work in fundamental ways: Cousot–Cousot's chaotic iteration uses join-based merges that preserve information [5]. Instead, we use coordinate-wise overwriting. Bertsekas's asynchronous methods assume contractions [2]. We use coordinate-wise overwriting with structural constraints (locality, bounded staleness) instead. Applications include parallel and distributed algorithms for the transitive closure, stable marriage, shortest paths, and fair division with subsidy problems. Our results provide the first exact least-fixed-point convergence guarantees for overwrite-based parallel updates without join operations or contraction assumptions.

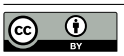
CCS Concepts

• **Theory of computation → Distributed algorithms**; **Parallel algorithms**.

Keywords

fixed points, distributive lattices, parallel algorithms, asynchronous computation, overwrite semantics, bounded staleness

ACM Reference Format:
Vijay K. Garg and Rohan Garg. 2026. Brief Announcement: Computing Least Fixed Points with Overwrite Semantics in Parallel and Distributed Systems. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 06–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3796701.3815932



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/2026/07
https://doi.org/10.1145/3796701.3815932

A full version of this brief announcement is available at https://arxiv.org/abs/2602.10486.

1 Introduction

Fixed-point computation [9, 10, 14] underlies logic, automata, and optimization, but its classical formulation iterates a single monotone function sequentially. Modern parallel and distributed systems instead use *overwrite* semantics: updates replace state rather than merge it, different workers maintain different components of the state, and communication delays force workers to operate on stale data. Classical fixed-point theorems do not directly address this setting. We model a computation on n cores or machines as the least *common* fixed point of n inflationary monotone functions, and prove convergence under three progressively relaxed synchronization models.

Theorem 2.1 treats interleaving parallelism in a finite lattice: atomic updates under a fair schedule converge to the least common fixed point. Theorem 3.1 drops atomicity: in a non-interleaving round-based model where multiple processes write concurrently and per-coordinate last-writer-wins, an additional assumption - *update-only-on-change*, that a processor writes coordinate i only when its value actually changes - restores convergence. Theorem 4.1 drops shared memory: each process P_i owns $G[i]$ and reads other coordinates with bounded staleness; if every f_i is *i-local* (writes only $G[i]$), then strongly fair execution still reaches the least common fixed point.

Our setting differs from prior work in two ways. Cousot & Cousot's chaotic iteration [5] guarantees convergence by merging updates with the lattice join, which preserves information; we use destructive overwrites instead. Baudet [1] and Bertsekas [2] handle multiple monotone operators under asynchrony but assume contractions; we do not. To our knowledge, this is the first exact least-fixed-point convergence result for coordinate-wise overwrite semantics that does not rely on join-based merging or contraction assumptions.

2 Fixed Points Under Interleaving Execution

Let $L \subseteq \mathbb{N}^n$ be a finite lattice ordered with the bottom element $\perp$. Let $\mathcal{F}$ be a set of m functions from L to L. We study the convergence of computation when $\{f_i \in \mathcal{F} | 1 \le i \le m\}$ are applied to some element $x \in L$ one at a time in some order. We assume that all the functions in $\mathcal{F}$ are inflationary and monotone.

A function $f : L \to L$ is *inflationary* if $\forall G \in L : \ f(G) \ge G$, and *monotone* if $\forall G, H \in L : \ G \le H \Rightarrow f(G) \le f(H)$.

We define the *least common fixed point* of $\mathcal{F}$ as $G^* = \min\{\, G \in L \mid \forall i, \ f_i(G) = G \,\}$. Observe that the set of fixed points for each

of the functions f_i is a sublattice of L. Hence, the set of common fixed points is an intersection of the sublattices and therefore also a sublattice Z. Since this sublattice Z is nonempty (the $\top$ element belongs to each sublattice because each function is inflationary), the operator min used above is well-defined.

Throughout, we will assume that the scheduler that applies the functions f_i is *fair*, i.e., it selects which f_i to apply, at each step infinitely often [7]. This leads to the following result.

Theorem 2.1 (Convergence to the least common fixed point under a fair schedule). *Let $\mathcal{F} = \{f_i : L \to L \mid i = 1, \ldots, m\}$ be a set of inflationary and monotone functions. Consider an execution starting from $G_0 = \bot$, where at each step t, a fair scheduler selects an index $i \in [m]$ and performs the atomic update $G_{t+1} = f_i(G_t)$. Then, the execution terminates at a state G^* which is the least common fixed point of $\mathcal{F}$.*

3 Fixed Point Theorem on Finite Distributive Lattices under Non–Interleaving Parallelism

We now consider the application of multiple f_i functions in parallel without the interleaving semantics. Theorem 2.1 assumes that at each step a *single* function f_i is applied atomically, i.e., $G_{t+1} = f_i(G_t)$. If instead multiple processes apply different functions *in parallel* using a naive "read–compute–write" rule, the computation can fail: the execution may not terminate at a common fixed point (even when all f_i are monotone and inflationary). In one parallel round, each process i reads the same old state G, computes $f_i(G)$, and then all processes write their results concurrently. In particular, a later write may overwrite earlier writes with stale data. Note that each process writes all coordinates of its computed vector, not just its own. Thus, if writes are not coordinated, the resulting state can be an arbitrary interleaving of coordinate-wise overwrites. Equivalently, the last-writer-wins on each coordinate; the outcome is a nondeterministic interleaving of writes. This behavior is prevented by the *update-only-on-change* assumption used in Theorem 3.1.

A processor satisfies *update-only-on-change* if it issues an update on $G[i]$ only when its value changes from what it read; equivalently, with $H = f(G)$, f writes coordinate i iff $G[i] \neq H[i]$. From an implementation perspective, this assumes that any f has access to the old value of its argument. When the new value is changed it can check if the value has changed and issues writes only on the changed values. We are assuming that the lattice L is finite and each component of G is simply an integer. Thus, we do not have to worry about checking equality for two floating-point numbers.

We define *non–interleaving semantics* as follows. Execution proceeds in sequential rounds $t = 0, 1, 2, \ldots$ with committed states G_t. In round t, a set of functions $\{f_i : i \in S_t\}$ is executed in parallel. Each function f_i may read coordinates of the shared state at different times during the round; hence some coordinates may be read before other functions' writes take effect.

The writes of the functions may overlap and can take effect in any order; if multiple functions write to the same coordinate, the final value at the end of the round is determined by the last write. The round completes once all started functions finish and their writes have taken effect, yielding the next committed state G_{t+1}. In this model, when a *read* overlaps with any number of *writes* it can return the value prior to all the writes or after any of the writes. Similarly, when multiple writes overlap, then the final value returned could be any of the *writes*. In particular, we do not require memory consistency guarantees such as *sequential consistency* [11] or linearizability [8].

A non–interleaving execution can be viewed as a computation where each round starts from a committed global state, and processes may observe partial effects of other writes within the same round, but no process ever observes a state older than the round's starting state. Although writes may overwrite each other, update-only-on-change ensures that no write can decrease a coordinate.

We also assume that our lattices are now distributive. If the parallel or distributed computation is modeled as a finite poset of width n, then the set of consistent global states forms a finite distributive lattice [3]. Let the poset P with n chains be written as $P_1, P_2, \ldots, P_n$. If the lattice L is generated from P, any element $G \in L$ can be viewed with $G[i]$ as the number of elements in P_i. Since we will be using this representation of the lattice, distributivity is assumed in the rest of the paper. For simplicity, we will assume that our lattice is simply a finite lattice of vectors of natural numbers such that each component $G[i]$ of the vector is a natural number between 0 and m.

Theorem 3.1 (Parallel convergence with non-interleaving semantics). *Let L be a finite distributive lattice of vectors $\mathbf{N}_m^n$ ordered with the usual bottom element $\bot = (0, 0, \ldots, 0)$ with $\leq$ defined as the component-wise comparison. For each $i \in \{1, \ldots, n\}$, let $f_i : L \to L$ be monotone and inflationary. Consider a parallel execution starting from $G_0 = \bot$ in which, at each round t, a scheduler selects a set $S_t \subseteq \{1, \ldots, n\}$ and the next state is defined by G_{t+1} which is the global state reached after any parallel execution of f_i on G_t where $i \in S_t$. Assume the scheduler is fair: each index i belongs to S_t for infinitely many rounds. The execution assumes the update-only-on-change model. Then, the execution terminates at a state G^* which is the least common fixed point of $\mathcal{F}$.*

Before we give the proof, we state the following Lemma.

Lemma 3.2 (Monotone round progress). *Consider a round t starting from the committed state G_t under the non–interleaving semantics model. Assume that each function f_i is inflationary. Assume further that updates are performed only when the computed value differs from the value read. Then for every coordinate j: (1) every write to coordinate j during round t writes a value greater than $G_t[j]$, (2) the committed value $G_{t+1}[j]$ satisfies $G_{t+1}[j] \geq G_t[j]$, and (3) $G_{t+1}[j] = G_t[j]$ if and only if no function writes to coordinate j during round t.*

Proof. (Lemma 3.2) Fix a coordinate j. Let a function f_i read value v for coordinate j during round t. Since we are in round t, $v \geq G_t[j]$. Since f_i is inflationary, the value v' computed by f_i for coordinate j satisfies $v' \geq v \geq G_t[j]$. Since f_i only writes if $v' > v$, every write to coordinate j during the round writes a value greater than $G_t[j]$, proving (1).

Because the committed value $G_{t+1}[j]$ is determined by one of the writes during the round (or remains unchanged if there are no writes), it follows that $G_{t+1}[j] \geq G_t[j]$, proving (2).

Finally, if $G_{t+1}[j] = G_t[j]$ and some function had written coordinate j, then by the update–only–on–change, rule that write

would have been strictly greater than the value read, and hence strictly greater than $G_t[j]$, contradicting $G_{t+1}[j] = G_t[j]$. □

Proof. (Theorem 3.1) We first show that the state sequence is monotone and stabilizes. Fix a round t and let S_t be the set selected by the scheduler. From Lemma 3.2, we get that $G_{t+1} \geq G_t$ componentwise. Therefore, the sequence $G_0 \leq G_1 \leq G_2 \leq \cdots$ is an ascending chain in the finite lattice L. Hence, it stabilizes: there exists T such that $G_T = G_{T+1} = \cdots$. Let $G^* := G_T$.

We now show that G^* is a common fixed point. We first show that for every i, $f_i(G^*) = G^*$. Suppose not. Then $f_i(G^*)[j] \neq G^*[j]$ for some $j \in [n]$. Because f_i is inflationary, this implies $f_i(G^*)[j] > G^*[j]$. By fairness, there exists a round $t \geq T$ with $i \in S_t$. Since $G_t = G^*$ for all $t \geq T$, the update rule gives $G_{t+1}[j] > G_t[j]$. If any write to coordinate j occurs during the round t, Lemma 3.2 guarantees that the committed value $G_{t+1}[j] > G_t[j]$ regardless of which functions's write wins.

This case arises when we have overlapping writes and a write by some other function may have won. Although overlapping writes may cause the committed value $G_{t+1}[j]$ to differ from $f_i(G^*)[j]$, Lemma 3.2 guarantees that any committed write to coordinate j during round t is strictly greater than $G_t[j]$. Hence $G_{t+1}[j] > G_t[j]$, contradicting the assumption that the execution has stabilized at G^*. Thus, G^* is a common fixed point.

We now show that G^* is the least common fixed point. Let Y be any common fixed point, i.e., $f_i(Y) = Y$ for all i. We prove by induction that $G_t \leq Y$ for all t. The base case holds since $G_0 = \perp \leq Y$. Now suppose that we are at round t and the state is G_t. Let S_t be the set of functions applied in the round t and G_{t+1} be the global state reached. Suppose a function f_i in S_t reads a vector X. Notice that if $G_t \leq Y$ and Y is a common fixed point, then all writes in round t are $\leq Y$ and so $X \leq Y$. Then by monotonicity, for all f_i, $f_i(X) \leq f_i(Y) = Y$. Therefore, the committed last-writer-wins state at end of round also satisfies $G_{t+1} \leq Y$. Hence, we obtain that $G^* \leq Y$. Thus, G^* is the least common fixed point of $\mathcal{F}$. □

4 Fixed Point Theorem on Finite Distributive Lattices under Distributed Computation

We now explore the program under the distributed computation model. Suppose that we are working in a distributed computing model where G is maintained in a distributed fashion. We will assume that there are n functions and the process i applies the function f_i. Thus, process P_i keeps the most up-to-date copy of $G[i]$. We assume that when process i applies the function f_i it changes only the component i that is locally stored. It can then relay this value to other processes by sending messages. A function $f_i : L \to L$ is *i-local* if $\forall G \in L,\ \forall j \neq i :\ f_i(G)[j] = G[j]$. Thus, if f_i is i-local, it may update only the ith coordinate and leaves all other coordinates unchanged (i.e., it does not write the value of other components). The function f_i may read the states from other processes, but can update only its own component.

When P_i applies f_i, it has the most recent $G[i]$ but other components of G may be old. Different processes may have different old values of the components for other processes. So, Theorem 3.1 may not hold.

To restore the convergence to the least fixed-point, we add the *bounded staleness* assumption and strengthen the notion of fairness.

A distributed system with n processes $P_1, \ldots, P_n$ maintaining G such that $G[i]$ is at P_i satisfies *bounded staleness* with parameter T if whenever P_i reads G, it gets the most recent $G[i]$ and a value of $G[j]$ that is at most T rounds old. A scheduler is *strongly fair* if every process is selected at least once in any window of τ consecutive rounds.

Observe that the strong fairness of the schedule does not imply bounded staleness. Strong fairness constrains which processes execute; bounded staleness constrains what information they observe. Strong fairness and bounded staleness are complementary: strong fairness with parameter τ ensures every process executes within τ rounds, while bounded staleness with parameter T ensures every process observes data at most T rounds old. Together, these conditions guarantee that after stabilization, every process eventually operates on the true fixed point.

Since computing f_i requires values from other processes, we keep the view of the G vector at each of the processes. We denote this view by $\widehat{G}^{(i)}$. However, from the perspective of the computation, we consider the vector G with i^{th} component at P_i. Thus, G is a distributed vector.

Theorem 4.1 (Distributed least fixed point theorem). *Let L be a distributive finite lattice of vectors $\mathbb{N}_m^n$. Let $\{f_i : L \to L\}_{i=1}^n$ be a set of monotone, inflationary, and i-local functions. Consider an execution $G_0 = \perp, G_1, G_2, \ldots$ generated as follows. At each round t, a strongly fair scheduler chooses a set $S_t \subseteq [n]$, and for each $i \in S_t$, process i computes its update using its view $\widehat{G}_t^{(i)} \in L$ satisfying bounded staleness. The coordinatewise commit rule is $G_{t+1}[i] := f_i(\widehat{G}_t^{(i)})[i]$ for $i \in S_t$, and $G_{t+1}[j] := G_t[j]$ for $j \notin S_t$.*

Then the execution terminates at the least common fixed point $G^* \in L$.

In Theorem 4.1, we observe that the system may have terminated (i.e., reached the least fixed point) but none of the processes may be aware of it. Hence, when we use the distributed version, we assume a termination detection algorithm that checks whether the computation has terminated. Any termination detection algorithm such as [6] is sufficient.

5 Related Work

Knaster–Tarski's theorem shows that any monotone function on a complete lattice has a least fixed point [14], and Kleene's theorem characterizes it constructively as $\mathrm{lfp}(F) = \bigvee_{k \geq 0} F^k(\perp)$ under ω-continuity [9]. Theorem 2.1 instead works with a *family* of functions under an arbitrary fair schedule, using finiteness and inflationarity in place of continuity.

Cousot & Cousot's *chaotic iteration* [4, 5] solves systems of monotone equations under fair, component-wise updates, but combines them with the lattice join, so it cannot accommodate the destructive overwrites we study. Conflict-free replicated data types (CRDTs) [12, 13] likewise resolve concurrent updates by join, falling under join-based merge rather than overwrite semantics.

Acknowledgments

This work was supported in part by the Cullen Trust Endowed Professorship.

References

[1] Gérard M. Baudet. 1978. Asynchronous Iterative Methods for Multiprocessors. *J. ACM* 25, 2 (1978), 226–244.

[2] Dimitri P. Bertsekas. 1983. Distributed Asynchronous Computation of Fixed Points. *Mathematical Programming* 27 (1983), 107–120.

[3] G. Birkhoff. 1967. *Lattice Theory*. American Mathematical Society, Providence, R.I. third edition.

[4] Patrick Cousot. 1977. *Asynchronous Iterative Methods for Solving a Fixed Point System of Monotone Equations in a Complete Lattice*. Research Report. IMAG, University of Grenoble.

[5] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, 238–252. doi:10.1145/512950.512973

[6] E. W. Dijkstra and C. S Scholten. 1980. Termination detection for diffusing computations. *Inform. Process. Lett.* 11, 4 (Aug. 1980), 1–4.

[7] Nissim Francez. 2012. *Fairness*. Springer Science & Business Media.

[8] M. P. Herlihy and J. M. Wing. 1990. Linearizability: A Correctness Condition for Atomic Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (July 1990), 463–492.

[9] Stephen C. Kleene. 1952. *Introduction to Metamathematics*. North-Holland.

[10] Bronislaw Knaster and Alfred Tarski. 1928. Un théorème sur les fonctions d'ensembles. *Annales de la Société Polonaise de Mathématique* 6 (1928), 133–134.

[11] L. Lamport. 1979. How to Make a Correct Multiprocess Program Execute Correctly on a Multiprocessor. *IEEE Trans. on Computers* 28, 9 (1979), 690–691.

[12] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. A comprehensive study of conflict-free replicated data types. *Research Report RR-7506, INRIA* (2011).

[13] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*. 386–400.

[14] Alfred Tarski. 1955. A lattice-theoretical fixpoint theorem and its applications. *Pacific J. Math.* 5, 2 (1955), 285–309.